```
# DirmuFX Signal Bot - kirim sinyal ke Telegram (jalan di GitHub Actions)
# Hanya pakai library bawaan Python, tidak perlu install apa pun.
import json, os, sys, time, urllib.request, urllib.parse, math

# ==================== PENGATURAN ====================
PAIRS = {  # nama : (simbol Binance, ukuran 1 pip, sesuaikan ke harga emas spot?)
    "XAUUSD": ("PAXGUSDT", 0.1, True),
    "EURUSD": ("EURUSDT", 0.0001, False),
    "BTCUSD": ("BTCUSDT", 1.0, False),
}
TFS = [5, 15]                        # M5 = scalping, M15 = intraday
HTF = {1: 5, 5: 15, 15: 60, 30: 60, 60: 240}
SL_PIPS = {1: 30, 5: 30, 15: 50, 30: 50, 60: 100, 240: 100}
RR = 2
SIG_MIN = 0.5      # ambang konfluensi sinyal
PRE_MIN = 0.4      # ambang "siap-siap"
ADX_MIN = 20
PRE_ALERT = os.environ.get("PRE_ALERT", "1") == "1"
TOOL_URL = "https://dirmufx.github.io/dirmufx.html"
IB_URL = "https://sc.myuserhub.com/?pt=224062"
BROKER = "Windsor Brokers"
STATE_FILE = "state.json"
BNIV = {1: "1m", 5: "5m", 15: "15m", 30: "30m", 60: "1h", 240: "4h"}
TFN = {1: "M1", 5: "M5", 15: "M15", 30: "M30", 60: "H1", 240: "H4"}
# ====================================================

TOKEN = os.environ.get("TELEGRAM_BOT_TOKEN", "").strip()
CHAT = os.environ.get("TELEGRAM_CHAT_ID", "").strip()
MODE = (sys.argv[1] if len(sys.argv) > 1 else os.environ.get("MODE", "run")).stri
NAN = float("nan")


def get_json(url, tries=3):
    for i in range(tries):
        try:
            req = urllib.request.Request(url, headers={"User-Agent": "dirmufx-bot
            with urllib.request.urlopen(req, timeout=20) as r:
                return json.loads(r.read().decode())
        except Exception as e:
            err = e
            time.sleep(2)
    raise RuntimeError(f"gagal ambil {url}: {err}")
```

```
def tg(method, **params):
    data = urllib.parse.urlencode(params).encode()
    req = urllib.request.Request(f"https://api.telegram.org/bot{TOKEN}/{method}",
    with urllib.request.urlopen(req, timeout=20) as r:
        return json.loads(r.read().decode())


def send(text):
    if not TOKEN or not CHAT:
        print("[TIDAK TERKIRIM - token/chat id kosong]\n" + text)
        return
    kb = json.dumps({"inline_keyboard": [[{"text": f"Buka akun {BROKER}", "url":
                                          [{"text": "Buka DirmuFX Trade", "url": T
    r = tg("sendMessage", chat_id=CHAT, text=text, parse_mode="HTML",
           disable_web_page_preview="true", reply_markup=kb)
    print("terkirim" if r.get("ok") else f"gagal kirim: {r}")


_spot = None
def gold_spot():
    global _spot
    if _spot is None:
        try:
            _spot = float(get_json("https://api.gold-api.com/price/XAU")["price"]
        except Exception as e:
            print("harga spot emas gagal:", e)
            _spot = 0.0
    return _spot


def candles(sym, tf):
    bn, _, spot = PAIRS[sym]
    j = None
    for host in ("https://data-api.binance.vision", "https://api.binance.com"):
        try:
            j = get_json(f"{host}/api/v3/klines?symbol={bn}&interval={BNIV[tf]}&l
            break
        except Exception as e:
            print("binance:", e)
    if not j or len(j) < 60:
        raise RuntimeError(f"data {sym} {TFN[tf]} kosong")
    d = [{"t": k[0], "o": float(k[1]), "h": float(k[2]), "l": float(k[3]), "c": f
    if spot and gold_spot() > 0:
        off = gold_spot() - d[-1]["c"]
        for x in d:
            for k in "ohlc":
                x[k] += off
```

```
    return d


# ---------------- indikator (sama dengan di tool) ----------------
def ema(a, p):
    k = 2 / (p + 1); o = [a[0]]
    for x in a[1:]:
        o.append(x * k + o[-1] * (1 - k))
    return o

def sma(a, p):
    o, s = [], 0.0
    for i, x in enumerate(a):
        s += x
        if i >= p: s -= a[i - p]
        o.append(s / p if i >= p - 1 else NAN)
    return o

def rsi(c, p=14):
    o = [NAN] * len(c); g = l = 0.0
    for i in range(1, len(c)):
        d = c[i] - c[i - 1]; up = max(d, 0); dn = max(-d, 0)
        if i <= p:
            g += up; l += dn
            if i == p:
                g /= p; l /= p; o[i] = 100 - 100 / (1 + g / (l or 1e-9))
        else:
            g = (g * (p - 1) + up) / p; l = (l * (p - 1) + dn) / p
            o[i] = 100 - 100 / (1 + g / (l or 1e-9))
    return o

def atr(d, p=14):
    tr = [d[0]["h"] - d[0]["l"]] + [max(x["h"] - x["l"], abs(x["h"] - d[i]["c"]),
    o, a = [], 0.0
    for i, t in enumerate(tr):
        if i < p:
            a += t
            if i == p - 1:
                a /= p; o.append(a)
            else:
                o.append(NAN)
        else:
            a = (a * (p - 1) + t) / p; o.append(a)
    return o

def macd_hist(c):
    e12, e26 = ema(c, 12), ema(c, 26)
```

```
    m = [a - b for a, b in zip(e12, e26)]; s = ema(m, 9)
    return [a - b for a, b in zip(m, s)]

def stoch(d, p=14):
    k = []
    for i, x in enumerate(d):
        if i < p - 1: k.append(NAN); continue
        hh = max(y["h"] for y in d[i - p + 1:i + 1]); ll = min(y["l"] for y in d[
        k.append(100 * (x["c"] - ll) / ((hh - ll) or 1e-9))
    ks = sma([50 if math.isnan(v) else v for v in k], 3)
    ds = sma([50 if math.isnan(v) else v for v in ks], 3)
    return ks, ds

def adx(d, p=14):
    tr = pd = md = 0.0; ax = NAN; dxs = []
    for i in range(1, len(d)):
        up = d[i]["h"] - d[i - 1]["h"]; dn = d[i - 1]["l"] - d[i]["l"]
        t = max(d[i]["h"] - d[i]["l"], abs(d[i]["h"] - d[i - 1]["c"]), abs(d[i]["
        p1 = up if (up > dn and up > 0) else 0; m1 = dn if (dn > up and dn > 0) e
        if i <= p: tr += t; pd += p1; md += m1
        else: tr = tr - tr / p + t; pd = pd - pd / p + p1; md = md - md / p + m1
        if i >= p:
            P = 100 * pd / (tr or 1e-9); M = 100 * md / (tr or 1e-9)
            dx = 100 * abs(P - M) / ((P + M) or 1e-9); dxs.append(dx)
            if len(dxs) == p: ax = sum(dxs) / p
            elif len(dxs) > p: ax = (ax * (p - 1) + dx) / p
    return ax

def boll(c, p=20, m=2):
    mid = sum(c[-p:]) / p
    sd = math.sqrt(sum((x - mid) ** 2 for x in c[-p:]) / p)
    return mid + m * sd, mid - m * sd

def pivot_p(d):
    day = lambda t: t // 86400000
    last = day(d[-1]["t"]); prev = [x for x in d if day(x["t"]) == last - 1]
    src = prev if len(prev) >= 3 else d[-min(len(d) - 1, 289):-1]
    H = max(x["h"] for x in src); L = min(x["l"] for x in src); C = src[-1]["c"]
    return (H + L + C) / 3

def analyse(d):
    c = [x["c"] for x in d]; n = len(c) - 1; close = c[n]
    e20, e50, e200 = ema(c, 20)[n], ema(c, 50)[n], ema(c, 200)[n]
    r = rsi(c)[n]; h = macd_hist(c); hist, hist1 = h[n], h[n - 1]
    ks, ds = stoch(d); k, dd = ks[n], ds[n]
    ax = adx(d); bu, bl = boll(c); pv = pivot_p(d)
    votes = []
```

```
    t = 1 if (close > e50 and e20 > e50) else -1 if (close < e50 and e20 < e50) e
    votes.append((2, t))
    votes.append((1, 1 if close > e200 else -1))
    votes.append((1.5, (1 if hist > hist1 else .5) if hist > 0 else (-1 if hist <
    votes.append((1, -.5 if r > 70 else .5 if r < 30 else 1 if r > 55 else -1 if
    votes.append((1, 1 if (k > dd and k < 80) else -1 if (k < dd and k > 20) else
    pb = (close - bl) / ((bu - bl) or 1e-9)
    votes.append((.5, .5 if pb < .05 else -.5 if pb > .95 else 0))
    votes.append((.5, 1 if close > pv else -1))
    norm = sum(w * s for w, s in votes) / sum(w for w, _ in votes)
    adxF = .85 if math.isnan(ax) else .65 if ax < 18 else .85 if ax < 25 else 1
    return {"norm": norm, "adx": ax, "rsi": r, "strength": round(min(1, abs(norm)


def side(norm, th):
    return "BUY" if norm >= th else "SELL" if norm <= -th else "WAIT"


def fmt(x, pip):
    dec = 2 if pip >= 0.1 else 3 if pip >= 0.01 else 5
    if pip >= 1: dec = 1
    return f"{x:,.{dec}f}"


def gold_closed():
    t = time.gmtime(); dy, hr = t.tm_wday, t.tm_hour  # Senin=0
    return (dy == 4 and hr >= 21) or dy == 5 or (dy == 6 and hr < 22)


def run():
    try:
        state = json.load(open(STATE_FILE))
    except Exception:
        state = {}
    cache = {}
    def data(sym, tf):
        if (sym, tf) not in cache: cache[(sym, tf)] = candles(sym, tf)
        return cache[(sym, tf)]

    for sym, (_, pip, spot) in PAIRS.items():
        if spot and gold_closed():
            print(sym, "pasar tutup, dilewati"); continue
        for tf in TFS:
            key = f"{sym}|{tf}"
            try:
                d = data(sym, tf); hd = data(sym, HTF[tf])
            except Exception as e:
```

```
                print(key, "error:", e); continue
            closed = d[:-1]; cand_t = closed[-1]["t"]; live = d[-1]["c"]
            a = analyse(closed); h = analyse(hd[:-1])
            htrend = side(h["norm"], 0.15)
            raw = side(a["norm"], SIG_MIN)
            sig = raw if (raw != "WAIT" and not math.isnan(a["adx"]) and a["adx"]
            st = state.get(key, {"last": "WAIT", "pre": ""})
            print(f"{key}: norm={a['norm']:.2f} adx={a['adx']:.1f} htf={htrend} -

            if sig != "WAIT" and st["last"] != sig:
                s = 1 if sig == "BUY" else -1; slp = SL_PIPS[tf]; dist = slp * pi
                sl, tp1, tp2 = live - s * dist, live + s * dist, live + s * dist
                icon = "🟢" if sig == "BUY" else "🔴"
                send(f"{icon} <b>SINYAL {sig} {sym}</b> · {TFN[tf]}\n\n"
                     f"Entry: <b>{fmt(live, pip)}</b>\n"
                     f"SL: <b>{fmt(sl, pip)}</b> ({slp} pips)\n"
                     f"TP1: <b>{fmt(tp1, pip)}</b> ({slp} pips)\n"
                     f"TP2: <b>{fmt(tp2, pip)}</b> ({slp * RR} pips)\n\n"
                     f"Konfluensi {a['strength']}% · ADX {a['adx']:.0f} · searah
                     f"Saat TP1 kena, geser SL ke entry.\n\n"
                     f"<i>Bukan saran keuangan. Selalu pakai stop loss.</i>")
                st["pre"] = ""
            elif sig == "WAIT" and PRE_ALERT and st["last"] == "WAIT":
                pre = side(a["norm"], PRE_MIN)
                ready = pre != "WAIT" and pre == htrend and not math.isnan(a["adx
                if ready and st.get("pre") != pre:
                    send(f"⏳ <b>SIAP-SIAP {pre} {sym}</b> · {TFN[tf]}\n"
                         f"Sinyal hampir terbentuk (konfluensi {a['strength']}%,
                         f"Tunggu konfirmasi sebelum entry.")
                    st["pre"] = pre
                elif abs(a["norm"]) < 0.3:
                    st["pre"] = ""
            st["last"] = sig; st["t"] = cand_t
            state[key] = st
    json.dump(state, open(STATE_FILE, "w"), indent=1, sort_keys=True)


if __name__ == "__main__":
    if not os.path.exists(STATE_FILE):
        json.dump({}, open(STATE_FILE, "w"))
    if MODE == "chatid":
        r = tg("getUpdates")
        chats = {}
        for u in r.get("result", []):
            m = u.get("message") or u.get("channel_post") or u.get("my_chat_membe
            ch = m.get("chat")
            if ch: chats[ch["id"]] = ch.get("title") or ch.get("username") or ch.
```

```
    print("CHAT ID YANG DITEMUKAN:" if chats else "Belum ada. Kirim satu pesa
    for i, t in chats.items(): print(f"  {i}  ->  {t}")
elif MODE == "test":
    send("✅ <b>Bot DirmuFX aktif.</b>\nSinyal XAUUSD, EURUSD, dan BTCUSD (M5
else:
    run()
```